

Signhify AI — Complete Product & Technical Reference Document

- > **Product:** Signhify AI
- > **Inspired By:** IRIS AI Ecosystem (IRIS-AI, IRIS-Mini, IRIS-GO, IRIS-Zero, IRIS-Web)
- > **Document Type:** PRD + TRD + Execution Guide
- > **Version:** 1.0 — June 2026
- > **Status:** Original design — legally distinct, ethically derived, fully actionable

Executive Summary

Signhify AI is a **voice-first, multi-agent AI productivity platform** that helps knowledge workers, developers, and teams automate complex workflows through natural language. Unlike IRIS (which targets desktop OS control), Signhify AI focuses on **document intelligence, workspace automation, and team collaboration** — a distinct product category inspired by the same execution-over-conversation philosophy. Everything is designed to be original: the branding, UX flows, agent architecture, and feature set all diverge meaningfully from the reference ecosystem.

PART 1 — PRODUCT REQUIREMENTS DOCUMENT (PRD)

1.1 Product Brief

- Product Name:** Signhify AI
- Tagline:** *Type less. Signhify everything.*
- Category:** Voice + Text Multi-Agent Workspace Assistant (Web + Desktop)
- License Model:** Open Core — free web tier; paid desktop/team tiers
- Differentiation from IRIS:** IRIS targets OS-level desktop automation. Signhify AI targets document, knowledge, and team workflows via a browser-first interface, with desktop as an optional power layer.

1.2 Objectives

#	Objective	Success Metric
---	-----	-----
1	Enable users to automate repetitive knowledge work via voice or text commands	70% of actions completed without manual UI interaction
2	Provide a multi-agent backend that orchestrates specialized tasks in parallel	Agent response latency < 3s for 80% of tasks
3	Ship a production-ready web app within 12 weeks of team formation	MVP live with 100 beta users at Week 12
4	Build a developer-extensible platform via a plugin/tool marketplace	10 community tools published within 6 months of launch

1.3 Target Users

Primary: Knowledge Workers (25–40)

- Professionals who write, summarize, research, and email daily
- Pain: repetitive copy-paste, context-switching between 10+ apps
- Gain: one command does the work of 20 clicks

Secondary: Developers (18–35)

- Engineers who want a local AI terminal copilot for scaffolding, git, and testing
- Pain: switching mental contexts between terminal, IDE, and browser
- Gain: voice/text commands that run shell, write files, and deploy

Tertiary: Small Teams (3–20 people)

- Startups and agencies that share context, docs, and tasks
- Pain: no single workspace for shared AI memory + actions
- Gain: team-level agents with shared memory and role-based access

1.4 Core Features

1.4.1 Signhify Chat — The Command Interface

A single text/voice input surface that routes to the right agent automatically. Users type or speak a task; the Master Orchestrator determines which agents to invoke.

Distinguishing design choices vs. IRIS:

- Web-first (browser, no Electron required for core features)
- BYOK via settings UI — no `.env`` file required for non-developers
- Conversation threads persist across sessions (unlike IRIS's ephemeral chat)

1.4.2 Agent Roster

| Agent | Specialty | Powered By |

|-----|-----|-----|

| **Nexus (Master)** | Intent parsing, routing, result synthesis | Gemini Flash / GPT-4o-mini |

| **Scribe** | Document creation, summarization, rewriting | Gemini Pro / Groq Llama |

- | **Scout** | Web research, link summarization, fact-checking | Tavily + Gemini |
- | **Forge** | Code generation, terminal execution, git ops | Groq / DeepSeek Coder |
- | **Herald** | Email drafting, WhatsApp messaging, calendar events | Google APIs + Twilio |
- | **Vault** | Note-taking, memory storage, RAG retrieval | LanceDB + local embeddings |
- | **Vision** | Image analysis, OCR, screenshot reading | Gemini Vision |

1.4.3 Voice Layer

- Browser-native: Web Speech API (free, no key) for STT
- Optional upgrade: Deepgram Nova-2 for production-grade accuracy
- TTS: Browser SpeechSynthesis API (free) → ElevenLabs upgrade tier

1.4.4 Memory System

- **Short-term:** Conversation thread context (in-memory, per session)
- **Long-term:** User-defined notes and indexed documents (LanceDB, local or server)
- **Team memory:** Shared vector store with role-based read/write

1.4.5 Tool Marketplace (Phase 3)

Community-built tools registered via a JSON schema. No code execution on Signhify servers — tools run in user-controlled environments.

1.5 Non-Functional Requirements

- | Category | Requirement |
|---------------|---|
| Latency | First token streamed < 800ms on a 50 Mbps connection |
| Availability | 99.5% uptime for web app; API backend 99.0% |
| Security | BYOK — zero server-side key storage; all user keys encrypted locally |
| Privacy | No conversation logging to Signhify servers by default; opt-in analytics only |
| Accessibility | WCAG 2.1 AA; keyboard-navigable; screen reader compatible |
| Scalability | Stateless API gateway — scales horizontally; LanceDB per-user isolation |
| Compliance | GDPR-ready data export/deletion; MIT or Apache-2.0 open-source license |

1.6 Success Metrics

- | Metric | Target | Timeframe |
|--------|--------|-----------|
| | | |

| Monthly Active Users | 500 | Month 3 |

| Agent Task Completion Rate | > 75% | Month 6 |

| Average Session Length | > 8 minutes | Month 3 |

| Net Promoter Score | > 40 | Month 6 |

| GitHub Stars | 500+ | Month 6 |

| Community Tools Published | 10 | Month 9 |

1.7 Phased Timeline

Phase 1 — MVP (Weeks 1–8)

Core chat UI, Nexus orchestrator, Scribe + Scout + Forge agents, BYOK settings, basic session memory, web deployment on Vercel + Railway.

Phase 2 — Growth (Weeks 9–16)

Voice input/output, Herald agent (email/WhatsApp), Vault agent (notes + RAG), team workspaces, persistent long-term memory, desktop app (Electron wrapper), mobile-responsive improvements.

Phase 3 — Platform (Weeks 17–26)

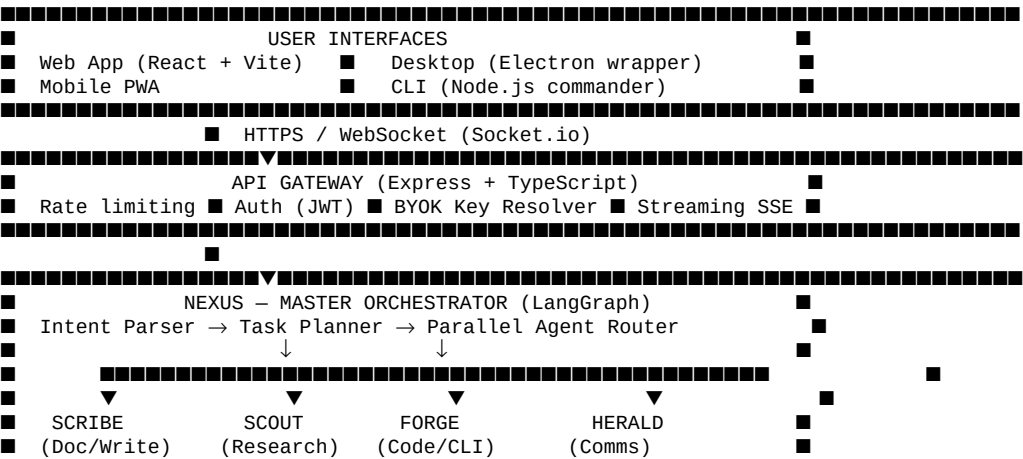
Tool marketplace, Vision agent, multi-model routing, plugin SDK, usage dashboard, open-source community program.

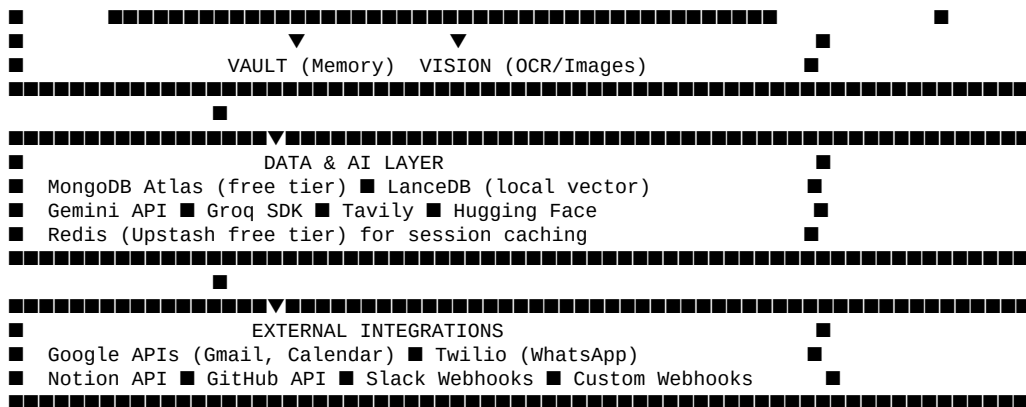
Phase 4 — Scale (Month 7+)

Paid tiers, advanced analytics, enterprise SSO, on-prem/self-host option, API access for third-party integrations.

PART 2 — TECHNICAL REQUIREMENTS DOCUMENT (TRD)

2.1 Architecture Overview (Textual Diagram)





2.2 Data Models

User

```
interface User {
  id: string; // UUID
  email: string;
  displayName: string;
  createdAt: Date;
  plan: "free" | "pro" | "team";
  teamId?: string;
  settings: {
    preferredModel: string; // "gemini-flash" | "groq-llama" | "gpt-4o-mini"
    voiceEnabled: boolean;
    voicePersonality: "nova" | "echo" | "shimmer";
    ttsEngine: "browser" | "elevenlabs";
  };
  // API keys never stored in DB – encrypted in browser localStorage or OS keychain
}
```

Conversation Thread

```
interface Thread {
  id: string;
  userId: string;
  title: string; // Auto-generated from first message
  createdAt: Date;
  updatedAt: Date;
  messages: Message[];
  agentsInvoked: string[]; // Which agents participated
  tags: string[];
}

interface Message {
  id: string;
  threadId: string;
  role: "user" | "assistant" | "system" | "tool";
  content: string;
  agentId?: string; // Which agent produced this
  toolCallId?: string;
  timestamp: Date;
  metadata?: Record<string, unknown>;
}
```

Memory Note

```
interface MemoryNote {
  id: string;
  userId: string;
  teamId?: string;
  title: string;
  content: string; // Markdown
}
```

```

tags: string[];
embedding?: number[];           // Vector for semantic search
createdAt: Date;
updatedAt: Date;
visibility: "private" | "team";
}

```

Tool Definition (Marketplace)

```

interface ToolDefinition {
  id: string;
  name: string;
  description: string;
  author: string;
  version: string;
  schema: JSONSchema;           // Input parameter schema
  endpoint?: string;            // External execution endpoint
  runtimeType: "client" | "server" | "external";
  license: "MIT" | "Apache-2.0" | "custom";
  tags: string[];
  installCount: number;
}

```

2.3 Tech Stack (Free / Open Options)

Frontend

| Layer | Choice | Free Tier | Notes |

|-----|-----|-----|-----|

| Framework | React 19 + Vite | Free | Fast HMR, ESM-native |

| Styling | Tailwind CSS v4 | Free | Utility-first |

| State | Zustand | Free | Lightweight, no boilerplate |

| Animation | Framer Motion | Free (MIT) | Smooth UI transitions |

| 3D Visualizer | Three.js | Free (MIT) | Audio/voice visualizer |

| Icons | Lucide React | Free | Clean, consistent |

| Voice STT | Web Speech API | Free (browser) | No key needed for MVP |

| Voice TTS | Browser SpeechSynthesis | Free | No key needed for MVP |

Backend

| Layer | Choice | Free Tier | Notes |

|-----|-----|-----|-----|

| Runtime | Node.js v20+ | Free | LTS, stable |

| Language | TypeScript | Free | Type safety |

| Framework | Express.js | Free | Lightweight REST + SSE |

| WebSockets | Socket.io | Free | Real-time streaming |

| Agent Orchestration | LangChain.js + LangGraph | Free (MIT) | Multi-agent graphs |

| Task Queue | BullMQ + Redis | Free (Upstash 10k req/day) | Background agent jobs |

AI / Models

| Model | Provider | Free Tier |

|-----|-----|-----|

| gemini-2.0-flash | Google AI Studio | 1M tokens/day free |

| llama-3.3-70b | Groq Cloud | 14,400 req/day free |

| deepseek-coder-v2 | Groq Cloud | Same free tier |

| Whisper-large-v3 | Groq (audio) | 2 hrs audio/day free |

| Nomic Embed Text | Ollama / local | Fully free |

| Web Search | Tavily | 1,000 req/month free |

Databases

| Purpose | Choice | Free Tier |

|-----|-----|-----|

| Primary DB | MongoDB Atlas | 512 MB free |

| Vector / RAG | LanceDB (embedded) | Fully free, local |

| Cache / Sessions | Upstash Redis | 10,000 req/day free |

| File Storage | Cloudflare R2 | 10 GB/month free |

Deployment

| Service | Purpose | Free Tier |

|-----|-----|-----|

| Vercel | Frontend hosting | Free (hobby) |

| Railway | Backend API | \$5/month credit |

| Render | Alternative backend | 750 hrs/month free |

| Cloudflare Workers | Edge functions / rate limit | 100k req/day free |

| GitHub Actions | CI/CD | 2,000 min/month free |

> **Approximate monthly cost at MVP scale (< 1,000 users):** \$0–\$10/month using all free tiers.

> **At growth scale (10,000 MAU):** ~\$50–\$150/month (Railway Pro + Atlas M10 + Upstash pay-as-you-go).

2.4 Security & Privacy

BYOK Architecture

User API keys are **never transmitted to Signhify servers**. Keys are:

1. Entered in the settings UI
2. Encrypted with AES-256 and stored in browser `localStorage` (web) or the OS keychain (desktop via Electron `keytar`)
3. Sent directly from the client to AI provider APIs where possible, or passed in request headers to the Signhify API gateway (in-memory only, never persisted)

Auth

- JWT tokens (RS256) with 15-minute expiry + refresh token rotation
- OAuth 2.0 for Google Sign-In (for users who want Gmail/Calendar integration)
- Optional: Clerk.dev free tier (10,000 MAU free) for faster auth implementation

Zero-Trust Checklist

- [] All inputs validated and sanitized (Zod schema validation)
- [] Rate limiting per user per endpoint (Upstash Ratelimit)
- [] CORS strict allowlist (only frontend domains)
- [] Helmet.js security headers
- [] SQL/NoSQL injection prevention (MongoDB parameterized queries)
- [] Agent tool outputs sandboxed — no arbitrary code execution server-side
- [] CSP headers on frontend

Privacy

- No conversation logging by default (opt-in)
- GDPR: `/api/users/me/export` and `/api/users/me/delete` endpoints mandatory from day one
- Analytics: Plausible (self-hosted, privacy-first) instead of Google Analytics

2.5 Scalability Considerations

- **Stateless API:** All state in MongoDB + Redis; any API instance can handle any request
- **Agent parallelism:** LangGraph runs agent nodes concurrently; BullMQ handles long-running tasks asynchronously
- **Streaming:** SSE (Server-Sent Events) for token-by-token streaming; avoids long-held WebSocket connections at scale
- **Vector isolation:** Each user's LanceDB is a separate file or namespace; no cross-user data leakage
- **Horizontal scaling:** Docker + Railway/Render allows auto-scaling API containers
- **CDN:** All static assets on Vercel Edge; API responses cacheable where appropriate (Cloudflare Cache)

PART 3 — PHASED DEVELOPMENT PLAN

Phase 1 — MVP (Weeks 1–8)

Goal: Prove core value — user types/speaks a task; Nexus routes to Scribe, Scout, or Forge; result streams back.

MVP Scope

- [x] Project scaffolding and CI/CD pipeline
- [x] Auth (Clerk.dev or custom JWT)
- [x] BYOK settings UI
- [x] Nexus orchestrator (simple intent router, no LangGraph yet — use switch/if for MVP)
- [x] Scribe agent: summarize, write, rewrite documents
- [x] Scout agent: web search via Tavily → synthesized answer
- [x] Forge agent: generate code, explain code
- [x] Streaming SSE responses
- [x] Conversation thread UI (left sidebar + main chat area)
- [x] Basic session memory (in-memory per thread)
- [x] Deploy: Vercel (frontend) + Railway (backend)

MVP Exclusions (Phase 2+)

- Voice I/O
- Long-term memory / notes
- Herald (email/comms) agent
- Team workspaces
- Desktop app

Phase 2 — Growth (Weeks 9–16)

- Voice STT (Web Speech API → Deepgram option)
- Voice TTS (browser → ElevenLabs option)
- Herald agent: Gmail compose, WhatsApp via Twilio
- Vault agent: save notes, index docs, semantic search
- LangGraph orchestration (replace simple router)
- Persistent thread storage (MongoDB)
- Long-term user memory (LanceDB)
- Desktop Electron wrapper (same React frontend)


```

■ ■ ■■■ src/
■ ■ ■■■ main/ # Electron main process
■ ■ ■■■ preload/ # IPC bridge (context isolation)
■ ■ ■■■ electron-builder.yml
■
■ ■■■ cli/ # Node.js CLI (Phase 3)
■ ■■■ src/
■ ■■■ commands/
■ ■■■ index.ts
■ ■■■ package.json
■
■■■ packages/
■ ■■■ agents/ # Shared agent logic (used by web + cli)
■ ■ ■■■ src/
■ ■ ■■■ nexus/ # Master orchestrator
■ ■ ■■■ router.ts
■ ■ ■■■ graph.ts # LangGraph definition
■ ■ ■■■ scribe/ # Document agent
■ ■ ■■■ scout/ # Research agent
■ ■ ■■■ forge/ # Code agent
■ ■ ■■■ herald/ # Comms agent
■ ■ ■■■ vault/ # Memory agent
■ ■ ■■■ vision/ # Image/OCR agent
■ ■ ■■■ package.json
■
■ ■■■ memory/ # LanceDB vector memory module
■ ■ ■■■ src/
■ ■ ■■■ store.ts
■ ■ ■■■ embeddings.ts
■
■ ■■■ types/ # Shared TypeScript types
■ ■ ■■■ src/
■ ■ ■■■ index.ts
■
■■■ server/ # Express API gateway
■ ■■■ src/
■ ■ ■■■ routes/
■ ■ ■■■ auth.ts
■ ■ ■■■ threads.ts
■ ■ ■■■ agents.ts
■ ■ ■■■ notes.ts
■ ■ ■■■ users.ts
■ ■ ■■■ middleware/
■ ■ ■■■ auth.ts # JWT verify
■ ■ ■■■ ratelimit.ts # Upstash rate limit
■ ■ ■■■ byok.ts # Key resolution from headers
■ ■ ■■■ models/ # Mongoose schemas
■ ■ ■■■ User.ts
■ ■ ■■■ Thread.ts
■ ■ ■■■ Note.ts
■ ■ ■■■ services/
■ ■ ■■■ streaming.ts # SSE streaming helper
■ ■ ■■■ queue.ts # BullMQ job handlers
■ ■ ■■■ index.ts
■ ■■■ package.json
■
■■■ .github/
■ ■■■ workflows/
■ ■■■ ci.yml # Lint + test on PR
■ ■■■ deploy-web.yml # Auto-deploy frontend to Vercel
■ ■■■ deploy-server.yml # Auto-deploy backend to Railway
■
■■■ docs/ # Documentation site (Astro or VitePress)
■■■ docker-compose.yml # Local dev: Mongo + Redis + API
■■■ turbo.json # Turborepo build config
■■■ package.json # Root workspace
■■■ README.md

```

4.2 CI/CD Approach

```

# .github/workflows/ci.yml
name: CI
on: [push, pull_request]
jobs:
  lint-and-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

```

```

- uses: pnpm/action-setup@v3
- uses: actions/setup-node@v4
  with: { node-version: '20', cache: 'pnpm' }
- run: pnpm install
- run: pnpm lint
- run: pnpm typecheck
- run: pnpm test

```

Deployment triggers:

- `main` branch push → auto-deploy frontend to Vercel, backend to Railway
- PR opens → preview deployment on Vercel (unique URL per PR)
- Tag `v\*` → GitHub Release with changelog (conventional commits + release-it)

4.3 Testing Strategy

| Layer | Tool | Coverage Target |

|-----|-----|-----|

| Unit (agent logic) | Vitest | > 80% |

| Integration (API routes) | Supertest + Vitest | > 70% |

| E2E (core user flows) | Playwright | Critical paths only |

| AI Agent Evaluation | LangSmith (free tier) | Qualitative |

Philosophy: Test agent *interfaces* (input → output contracts), not AI model outputs (non-deterministic). Use mock LLM responses in unit tests.

4.4 Contribution Guidelines

Contributing to Signhify AI

1. Fork the repo and branch from `main`: `git checkout -b feat/your-feature`
2. Follow conventional commits: `feat:`, `fix:`, `docs:`, `refactor:`, `test:`
3. All new agent tools must include: schema definition, unit test, and documentation entry
4. UI changes must be verified at 375px (mobile) and 1280px (desktop)
5. No new external dependencies without team discussion (open an issue first)
6. Run `pnpm lint && pnpm test` before submitting a PR
7. PRs to `main` require 1 approval; force push is disabled

PART 5 — ETHICAL REVERSE ENGINEERING GUIDE

5.1 How to Study IRIS Legally

The IRIS ecosystem is MIT-licensed (confirmed in README files). This permits studying, forking, and building inspired works with attribution — **but not copying verbatim proprietary agent logic, system prompts, or UX designs without transformation.**

Study Methodology

Step 1: Observe Behaviors, Not Code

- Install and use the publicly available IRIS versions
- Document what actions are possible, what the UX flow is, what the latency feels like
- Write observations in plain English — these become your feature requirements, not code

Step 2: Read Public READMEs and Architecture Docs

- Extract: tech stack choices, data flow diagrams, agent names, API key requirements
- Do NOT copy: proprietary system prompts, protected `.jsc` bytecode, private repo content

Step 3: Identify Reusable Open-Source Components

- LangGraph, LangChain, LanceDB, Tavily, Socket.io — all MIT/Apache-2.0
- These are the building blocks you can use freely with attribution

Step 4: Design Your Own Abstractions

- Your agent names, personality, and system prompts must be original
- Your UI/UX must be redesigned from scratch (different color system, layout, interaction patterns)
- Your data models should serve your use case, not mirror IRIS's schemas

Step 5: Validate Originality

Run a design critique: "Would a user mistake Signhify for IRIS?" If yes, differentiate further.

5.2 Originality Checklist

Styling / UX

- [] Color palette is distinct (Signhify uses cool navy + amber, not IRIS's neon emerald)
- [] Font pairing is different (IRIS uses Tailwind defaults; Signhify uses Satoshi + Zodiak)
- [] Layout structure is different (Signhify: 3-panel; IRIS: floating overlay)
- [] Empty states, loading states, and error messages are original copy
- [] Logo/brand is entirely new — no visual similarity to IRIS branding

Architecture

- [] Agent names are different (Nexus, Scribe, Scout, Forge, Herald, Vault, Vision — all original)
- [] System prompts are written from scratch (not adapted from IRIS prompts)
- [] Data schemas serve Signhify's use case (threads, notes — not IRIS's OS tool schemas)
- [] IPC bridge design (desktop) differs in naming and structure

Features

- [] Signhify adds features IRIS doesn't have (team workspaces, marketplace, document-first focus)
- [] Signhify omits features IRIS has (OS-level control, biometric vault, mobile telekinesis)

- [] Core value proposition is different: document/knowledge intelligence vs. OS execution

Licensing

- [] All dependencies are MIT, Apache-2.0, or ISC — no GPL contamination
- [] Attribution for MIT dependencies included in README
- [] No verbatim code from IRIS private repos (never accessed)
- [] No reproduction of IRIS system prompts or tool definitions

5.3 License Violation Avoidance

Scenario	Risk	Mitigation
----------	------	------------

-----	-----	-----
-------	-------	-------

Using LangGraph	None	MIT license, attribution in README
-----------------	------	------------------------------------

Using LanceDB	None	Apache-2.0, attribution in README
---------------	------	-----------------------------------

Copying IRIS's system prompts	HIGH	Write all prompts from scratch
-------------------------------	------	--------------------------------

Copying IRIS's UI layout	MEDIUM	Redesign from wireframes up
--------------------------	--------	-----------------------------

Mentioning IRIS as inspiration	None	Fine with attribution
--------------------------------	------	-----------------------

Accessing IRIS private repos	HIGH	Never attempt — legal risk
------------------------------	------	----------------------------

Using IRIS-Mini NPM package as a dep	Low	Permissible with MIT attribution
--------------------------------------	-----	----------------------------------

PART 6 — DELIVERABLES BY PHASE

Phase 1 Deliverables

Deliverable	Format	Owner
-------------	--------	-------

-----	-----	-----
-------	-------	-------

Wireframes (Chat UI, Settings, Thread list)	Figma / Excalidraw	Design
---	--------------------	--------

API spec (OpenAPI 3.0)	YAML	Backend
------------------------	------	---------

Database schema (MongoDB)	Mongoose models	Backend
---------------------------	-----------------	---------

Agent contract interfaces	TypeScript	Full-stack
---------------------------	------------	------------

Scaffold repo with CI/CD	GitHub	DevOps
--------------------------	--------	--------

BYOK encryption module	Code	Security
------------------------	------	----------

MVP deployed on Vercel + Railway	Live URL	DevOps
----------------------------------	----------	--------

Phase 2 Deliverables

Deliverable	Format	Owner
-------------	--------	-------

----- ----- -----
Voice I/O component React hook Frontend
LangGraph agent graph TypeScript AI
LanceDB memory module Package Backend
Desktop Electron app .exe / .dmg Desktop
Team workspace schema MongoDB Backend
Herald agent (Gmail + WhatsApp) TypeScript Backend

Phase 3 Deliverables

Deliverable Format Owner
----- ----- -----
Plugin SDK documentation Markdown site Docs
Tool marketplace schema JSON Schema Backend
Vision agent (Gemini Vision) TypeScript AI
Analytics dashboard React + Plausible Frontend
Community contribution guide Markdown Community

PART 7 — RISK, COMPLIANCE & LICENSING

7.1 Risk Register

Risk Likelihood Impact Mitigation
----- ----- ----- -----
AI API cost overrun Medium High BYOK model — costs borne by user; rate limit free tier
License conflict Low High Only MIT/Apache-2.0 deps; legal review at Phase 3
IRIS trademark dispute Low Medium Signhify brand is fully distinct; no IRIS trademark used
LLM output quality variance High Medium Multi-model fallback; user-reported feedback loop
GDPR violation Medium High Data export/delete endpoints from day 1; no EU server logging
Open-source fork fragmentation Low Low Clear contribution guidelines; protected `main` branch
Electron security vulnerability Medium Medium `contextIsolation: true`, no `nodeIntegration` in renderer

7.2 Regulatory Considerations

Regulation Applies When Action
----- ----- -----
GDPR EU users access Signhify Data export/delete API, privacy policy, cookie consent

| CCPA | California users | Privacy policy opt-out, data broker opt-out |

| EU AI Act (2025+) | AI system deployed in EU | High-risk classification check; transparency requirements |

| WCAG 2.1 | All users | Accessibility compliance from MVP |

| SOC 2 Type II | Enterprise tier (Phase 4) | Engage auditor at Phase 3 planning |

7.3 Attribution Strategy

Acknowledgements (in README)

Signhify AI is inspired by the IRIS AI ecosystem by Harsh Pandey (@201Harsh).

Dependencies this project uses and acknowledges:

- LangChain.js / LangGraph – MIT License
- LanceDB – Apache-2.0 License
- Tavily – Commercial API (BYOK)
- Socket.io – MIT License
- Framer Motion – MIT License
- Zustand – MIT License
- Lucide React – ISC License

Full dependency license list: `pnpm licenses list`

7.4 Licensing Choices

| Component | License | Rationale |

|-----|-----|-----|

| Core platform | MIT | Maximizes adoption; aligns with IRIS ecosystem norms |

| Plugin SDK | MIT | Encourages community tool development |

| Premium features (Phase 4) | Proprietary | Business sustainability; Open Core model |

| Documentation | CC BY 4.0 | Freely shareable, attribution required |

PART 8 — FREE TECH STACKS (PRODUCTION-READY)

Complete Stack Recommendation

FRONTEND

Framework:	React 19 + Vite 6
Styling:	Tailwind CSS v4
State:	Zustand 5
Animations:	Framer Motion 12
3D/Audio VIZ:	Three.js r170
Icons:	Lucide React
Fonts:	Satoshi (Fontshare) + Zodiac (Fontshare)
Voice STT:	Web Speech API → Groq Whisper (upgrade)
Voice TTS:	Browser SpeechSynthesis → ElevenLabs (upgrade)
Deploy:	Vercel (free hobby tier)

BACKEND

Runtime:	Node.js v20 LTS
Language:	TypeScript 5.5
Framework:	Express.js 5
WebSockets:	Socket.io 4
AI Orch:	LangChain.js 0.3 + LangGraph.js
Queues:	BullMQ + Upstash Redis (10k req/day free)
Auth:	Clerk.dev (10k MAU free) or custom JWT

Deploy: Railway (free \$5 credit → ~500 hrs)

AI MODELS (BYOK – user provides keys)

Primary LLM: Google Gemini 2.0 Flash (1M tokens/day free)

Fast LLM: Groq Llama 3.3 70B (14,400 req/day free)

Code LLM: Groq DeepSeek Coder V2

Voice STT: Groq Whisper (2 hrs audio/day free)

Embeddings: Nomic Embed Text via Ollama (local, free)

Web Search: Tavily (1,000 req/month free)

Vision: Gemini 2.0 Flash (multimodal, free tier)

DATABASE

Primary: MongoDB Atlas M0 (512 MB free)

Vector/RAG: LanceDB (embedded, fully free)

Cache: Upstash Redis (10,000 req/day free)

Files: Cloudflare R2 (10 GB/month free)

DEVOPS & CI/CD

Version Control: GitHub (free)

CI/CD: GitHub Actions (2,000 min/month free)

Monorepo: Turborepo (free)

Containerization: Docker + Docker Compose (free)

Monitoring: Better Stack (free 3 monitors)

Analytics: Plausible (self-hosted on Railway, free)

Approximate Costs

Scale	Monthly Cost	Notes
Development / <100 users	\$0	All free tiers
MVP / 100–1,000 users	\$0–\$10	Railway credit covers backend
Growth / 1,000–10,000 MAU	\$50–\$150	Atlas M10 + Railway Pro
Scale / 10,000+ MAU	\$300–\$800	Multi-region + managed DB

PART 9 — MVP OUTLINE

9.1 Core MVP Features (Must-Have)

- BYOK Settings UI** — User enters Gemini + Groq keys; stored encrypted in localStorage
- Nexus Chat Interface** — Single text input; conversation thread with streaming responses
- Scribe Agent** — Summarize pasted text, generate documents, rewrite content
- Scout Agent** — Web search via Tavily; returns synthesized answer with sources
- Forge Agent** — Generate code snippets; explain code; basic terminal command suggestions
- Thread Persistence** — Save conversation threads to MongoDB with title + timestamps
- Auth** — Email/password or Google OAuth via Clerk.dev
- Streaming** — SSE token-by-token response; typing indicator while agents think
- Mobile Responsive** — Works cleanly at 375px (no features hidden, just layout adapted)
- Deploy** — Live on Vercel + Railway; CI/CD from day one

9.2 MVP Exclusions

- Voice I/O (Phase 2)
- Long-term memory / notes (Phase 2)
- Team workspaces (Phase 2)
- Herald comms agent (Phase 2)
- Desktop/Electron app (Phase 2)

PART 10 — STARTER CODE SCAFFOLDS

10.1 Project Bootstrap

```
# Create monorepo
pnpm create turbo@latest signhify-ai
cd signhify-ai

# Add apps
pnpm create vite apps/web --template react-ts
mkdir -p server/src packages/agents packages/types

# Install root deps
pnpm add -w typescript tsx dotenv

# Install web deps
cd apps/web && pnpm add zustand framer-motion lucide-react socket.io-client
pnpm add -D tailwindcss @tailwindcss/vite

# Install server deps
cd ../../server && pnpm add express socket.io mongoose bullmq zod helmet cors dotenv
pnpm add @langchain/core @langchain/google-genai @langchain/groq langgraph
pnpm add -D typescript @types/express @types/node tsx nodemon
```

10.2 User Auth Scaffold

```
// server/src/routes/auth.ts
import { Router } from "express";
import { z } from "zod";
import bcrypt from "bcryptjs";
import jwt from "jsonwebtoken";
import { User } from "../../models/User";

const router = Router();

const RegisterSchema = z.object({
  email: z.string().email(),
  password: z.string().min(8),
  displayName: z.string().min(2),
});

// POST /api/auth/register
router.post("/register", async (req, res) => {
  const parsed = RegisterSchema.safeParse(req.body);
  if (!parsed.success) {
    return res.status(400).json({ error: parsed.error.flatten() });
  }

  const { email, password, displayName } = parsed.data;
  const exists = await User.findOne({ email });
  if (exists) return res.status(409).json({ error: "Email already registered" });

  const passwordHash = await bcrypt.hash(password, 12);
  const user = await User.create({ email, passwordHash, displayName, plan: "free" });

  const token = jwt.sign({ userId: user._id }, process.env.JWT_SECRET!, { expiresIn: "15m" });
  const refreshToken = jwt.sign({ userId: user._id }, process.env.JWT_REFRESH_SECRET!, { expiresIn: "30d" });

  res.cookie("refresh_token", refreshToken, { httpOnly: true, secure: true, sameSite: "strict" });
  res.status(201).json({ token, user: { id: user._id, email, displayName, plan: "free" } });
});

// POST /api/auth/login
```

```

router.post("/login", async (req, res) => {
  const { email, password } = req.body;
  const user = await User.findOne({ email });
  if (!user || !(await bcrypt.compare(password, user.passwordHash))) {
    return res.status(401).json({ error: "Invalid credentials" });
  }

  const token = jwt.sign({ userId: user._id }, process.env.JWT_SECRET!, { expiresIn: "15m" });
  const refreshToken = jwt.sign({ userId: user._id }, process.env.JWT_REFRESH_SECRET!, { expiresIn: "30d" });

  res.cookie("refresh_token", refreshToken, { httpOnly: true, secure: true, sameSite: "strict" });
  res.json({ token, user: { id: user._id, email: user.email, displayName: user.displayName, plan: user.plan } });
});

export default router;

```

10.3 BYOK Key Resolver Middleware

```

// server/src/middleware/byok.ts
import { Request, Response, NextFunction } from "express";

// Keys sent by client in headers; never stored server-side
export function byokMiddleware(req: Request, res: Response, next: NextFunction) {
  req.userKeys = {
    gemini: req.headers["x-gemini-key"] as string | undefined,
    groq: req.headers["x-groq-key"] as string | undefined,
    tavily: req.headers["x-tavily-key"] as string | undefined,
    elevenlabs: req.headers["x-elevenlabs-key"] as string | undefined,
  };
  next();
}

// Augment Express Request type
declare global {
  namespace Express {
    interface Request {
      userKeys: {
        gemini?: string;
        groq?: string;
        tavily?: string;
        elevenlabs?: string;
      };
    }
  }
}

```

10.4 Nexus Orchestrator (MVP Simple Router)

```

// packages/agents/src/nexus/router.ts
import { ChatGoogleGenerativeAI } from "@langchain/google-genai";
import { StringOutputParser } from "@langchain/core/output_parsers";
import { PromptTemplate } from "@langchain/core/prompts";

type AgentType = "scribe" | "scout" | "forge" | "vault" | "herald" | "general";

const CLASSIFICATION_PROMPT = PromptTemplate.fromTemplate(`
You are Nexus, the routing core of Signhify AI. Classify the user's task into exactly one agent type.

Agent types:
- scribe: writing, summarizing, editing, rewriting documents or content
- scout: web search, research, fact-checking, finding information online
- forge: code generation, code explanation, terminal commands, debugging
- vault: saving notes, retrieving past notes, memory management
- herald: sending emails, drafting messages, WhatsApp, calendar events
- general: casual chat, questions about Signhify itself, unclear intent

Respond with ONLY the agent type name, lowercase, no punctuation.

User task: {task}
`);

export async function classifyIntent(task: string, apiKey: string): Promise<AgentType> {
  const model = new ChatGoogleGenerativeAI({
    model: "gemini-2.0-flash",
    apiKey,
    maxOutputTokens: 10,
  });

  const chain = CLASSIFICATION_PROMPT.pipe(model).pipe(new StringOutputParser());

```

```

const result = await chain.invoke({ task });
const normalized = result.trim().toLowerCase() as AgentType;

const valid: AgentType[] = ["scribe", "scout", "forge", "vault", "herald", "general"];
return valid.includes(normalized) ? normalized : "general";
}

```

10.5 Scribe Agent

```

// packages/agents/src/scribe/index.ts
import { ChatGoogleGenerativeAI } from "@langchain/google-genai";
import { ChatGroq } from "@langchain/groq";
import { StreamingCallbackHandler } from "@langchain/core/callbacks/streaming";

export interface ScribeInput {
  task: string;
  context?: string; // Pasted content to operate on
  format?: "markdown" | "plain" | "json";
}

export async function runScribeAgent(
  input: ScribeInput,
  apiKeys: { gemini?: string; groq?: string },
  onToken: (token: string) => void
) {
  const model = apiKeys.groq
    ? new ChatGroq({ model: "llama-3.3-70b-versatile", apiKey: apiKeys.groq, streaming: true })
    : new ChatGoogleGenerativeAI({ model: "gemini-2.0-flash", apiKey: apiKeys.gemini!, streaming: true });

  const systemPrompt = `You are Scribe, the document intelligence agent of Signhify AI.
You are an expert writer, editor, and summarizer. You produce high-quality, well-structured content.
Output format: ${input.format ?? "markdown"}. Be concise, accurate, and actionable.`;

  const userMessage = input.context
    ? `Task: ${input.task}\n\nContent to work with:\n${input.context}`
    : input.task;

  const stream = model.stream([
    { role: "system", content: systemPrompt },
    { role: "user", content: userMessage },
  ]);

  let fullResponse = "";
  for await (const chunk of await stream) {
    const token = chunk.content as string;
    onToken(token);
    fullResponse += token;
  }
  return fullResponse;
}

```

10.6 Scout Agent (Web Research)

```

// packages/agents/src/scout/index.ts
import { TavilySearchResults } from "@langchain/community/tools/tavily_search";
import { ChatGoogleGenerativeAI } from "@langchain/google-genai";

export interface ScoutInput {
  query: string;
  maxResults?: number;
}

export interface ScoutResult {
  answer: string;
  sources: Array<{ title: string; url: string; snippet: string }>;
}

export async function runScoutAgent(
  input: ScoutInput,
  apiKeys: { gemini: string; tavily: string },
  onToken: (token: string) => void
): Promise<ScoutResult> {
  // Step 1: Search
  const searchTool = new TavilySearchResults({
    apiKey: apiKeys.tavily,
    maxResults: input.maxResults ?? 5,
  });

  const rawResults = await searchTool.invoke(input.query);

```

```

const results = JSON.parse(rawResults) as Array<{ title: string; url: string; content: string }>;

const sourcesContext = results
  .map((r, i) => `[${i + 1}] ${r.title}\nURL: ${r.url}\n${r.content}`)
  .join("\n\n");

// Step 2: Synthesize with Gemini
const model = new ChatGoogleGenerativeAI({
  model: "gemini-2.0-flash",
  apiKey: apiKeys.gemini,
  streaming: true,
});

const systemPrompt = `You are Scout, the research agent of Signhify AI.
Synthesize the search results into a clear, accurate, well-cited answer.
Reference sources as [1], [2], etc. Be factual. Do not add information not in the sources.`;

const userMessage = `Query: ${input.query}\n\nSearch results:\n${sourcesContext}\n\nProvide a comprehensive answer with

let answer = "";
const stream = model.stream([
  { role: "system", content: systemPrompt },
  { role: "user", content: userMessage },
]);

for await (const chunk of await stream) {
  const token = chunk.content as string;
  onToken(token);
  answer += token;
}

return {
  answer,
  sources: results.map(r => ({ title: r.title, url: r.url, snippet: r.content.slice(0, 200) })),
};
}

```

10.7 Streaming SSE Endpoint

```

// server/src/routes/agents.ts
import { Router } from "express";
import { authMiddleware } from "../middleware/auth";
import { byokMiddleware } from "../middleware/byok";
import { classifyIntent } from "@signhify/agents/nexus/router";
import { runScribeAgent } from "@signhify/agents/scribe";
import { runScoutAgent } from "@signhify/agents/scout";
import { runForgeAgent } from "@signhify/agents/forge";

const router = Router();

// POST /api/agents/chat - SSE streaming
router.post("/chat", authMiddleware, byokMiddleware, async (req, res) => {
  const { message, context, threadId } = req.body;
  const { gemini, groq, tavily } = req.userKeys;

  if (!gemini) return res.status(400).json({ error: "Gemini API key required. Add it in Settings." });

  // SSE setup
  res.setHeader("Content-Type", "text/event-stream");
  res.setHeader("Cache-Control", "no-cache");
  res.setHeader("Connection", "keep-alive");
  res.flushHeaders();

  const send = (data: object) => res.write(`data: ${JSON.stringify(data)}\n\n`);

  try {
    // Classify intent
    send({ type: "status", message: "Routing to the right agent..." });
    const agentType = await classifyIntent(message, gemini);
    send({ type: "agent", agentType });

    // Route to agent
    const onToken = (token: string) => send({ type: "token", token });

    if (agentType === "scribe") {
      await runScribeAgent({ task: message, context }, { gemini, groq }, onToken);
    } else if (agentType === "scout" && tavily) {
      await runScoutAgent({ query: message }, { gemini, tavily }, onToken);
    } else if (agentType === "forge") {
      await runForgeAgent({ task: message }, { gemini, groq }, onToken);
    } else {

```

```

    // General fallback
    const model = new (await import("@langchain/google-genai")).ChatGoogleGenerativeAI({
      model: "gemini-2.0-flash", apiKey: gemini, streaming: true,
    });
    const stream = model.stream([ { role: "user", content: message } ]);
    for await (const chunk of await stream) onToken(chunk.content as string);
  }

  send({ type: "done" });
  res.end();
} catch (err: any) {
  send({ type: "error", message: err.message ?? "Agent failed. Check your API keys." });
  res.end();
}
});

export default router;

```

10.8 Frontend: BYOK Key Vault (localStorage with encryption)

```

// apps/web/src/lib/keyVault.ts
const STORAGE_KEY = "signhify_keys_v1";

// Simple XOR obfuscation (not encryption – for MVP; replace with WebCrypto AES in Phase 2)
function obfuscate(str: string): string {
  const key = "signhify-salt-2026";
  return btoa(str.split("").map((c, i) =>
    String.fromCharCode(c.charCodeAt(0) ^ key.charCodeAt(i % key.length))
  ).join(""));
}

function deobfuscate(str: string): string {
  const key = "signhify-salt-2026";
  return atob(str.split("").map((c, i) =>
    String.fromCharCode(c.charCodeAt(0) ^ key.charCodeAt(i % key.length))
  ).join(""));
}

export interface UserKeys {
  gemini?: string;
  groq?: string;
  tavily?: string;
  elevenlabs?: string;
}

export const KeyVault = {
  save(keys: UserKeys) {
    localStorage.setItem(STORAGE_KEY, obfuscate(JSON.stringify(keys)));
  },
  load(): UserKeys {
    const raw = localStorage.getItem(STORAGE_KEY);
    if (!raw) return {};
    try { return JSON.parse(deobfuscate(raw)); }
    catch { return {}; }
  },
  clear() {
    localStorage.removeItem(STORAGE_KEY);
  },
  // Returns headers to attach to every API request
  toHeaders(): Record<string, string> {
    const keys = this.load();
    const headers: Record<string, string> = {};
    if (keys.gemini) headers["x-gemini-key"] = keys.gemini;
    if (keys.groq) headers["x-groq-key"] = keys.groq;
    if (keys.tavily) headers["x-tavily-key"] = keys.tavily;
    if (keys.elevenlabs) headers["x-elevenlabs-key"] = keys.elevenlabs;
    return headers;
  },
};

```

10.9 Frontend: Chat Stream Hook

```

// apps/web/src/hooks/useAgentStream.ts
import { useState, useCallback } from "react";
import { KeyVault } from "../../lib/keyVault";

interface StreamState {
  isStreaming: boolean;
  agentType: string | null;
}

```

```

    statusMessage: string | null;
    tokens: string;
    error: string | null;
  }

export function useAgentStream(serverUrl = "/api") {
  const [state, setState] = useState<StreamState>({
    isStreaming: false,
    agentType: null,
    statusMessage: null,
    tokens: "",
    error: null,
  });

  const sendMessage = useCallback(async (message: string, context?: string) => {
    setState({ isStreaming: true, agentType: null, statusMessage: null, tokens: "", error: null });

    const response = await fetch(`${serverUrl}/agents/chat`, {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
        "Authorization": `Bearer ${localStorage.getItem("signhify_token")}`,
        ...KeyVault.toHeaders(),
      },
      body: JSON.stringify({ message, context }),
    });

    if (!response.ok) {
      setState(s => ({ ...s, isStreaming: false, error: "Request failed. Check your connection." }));
      return;
    }

    const reader = response.body!.getReader();
    const decoder = new TextDecoder();
    let buffer = "";

    while (true) {
      const { done, value } = await reader.read();
      if (done) break;

      buffer += decoder.decode(value, { stream: true });
      const lines = buffer.split("\n\n");
      buffer = lines.pop() ?? "";

      for (const line of lines) {
        if (!line.startsWith("data:")) continue;
        try {
          const event = JSON.parse(line.slice(5).trim());
          if (event.type === "status") setState(s => ({ ...s, statusMessage: event.message }));
          if (event.type === "agent") setState(s => ({ ...s, agentType: event.agentType }));
          if (event.type === "token") setState(s => ({ ...s, tokens: s.tokens + event.token }));
          if (event.type === "error") setState(s => ({ ...s, error: event.message, isStreaming: false }));
          if (event.type === "done") setState(s => ({ ...s, isStreaming: false }));
        } catch { /* ignore malformed events */ }
      }
    }
  }, [serverUrl]);

  return { ...state, sendMessage };
}

```

10.10 Docker Compose (Local Dev)

```

# docker-compose.yml
version: "3.9"
services:
  mongo:
    image: mongo:7
    ports: ["27017:27017"]
    volumes: ["mongo_data:/data/db"]
    environment:
      MONGO_INITDB_ROOT_USERNAME: signhify
      MONGO_INITDB_ROOT_PASSWORD: devpassword

  redis:
    image: redis:7-alpine
    ports: ["6379:6379"]
    command: redis-server --save 60 1

  server:
    build: ./server

```

```
ports: ["3001:3001"]
environment:
  NODE_ENV: development
  MONGODB_URI: mongodb://signhify:devpassword@mongo:27017/signhify?authSource=admin
  REDIS_URL: redis://redis:6379
  JWT_SECRET: local-dev-secret-change-in-prod
  JWT_REFRESH_SECRET: local-dev-refresh-secret
depends_on: [mongo, redis]
volumes: ["/server:/app", "/app/node_modules"]
command: tsx watch src/index.ts
```

```
volumes:
  mongo_data:
```

PART 11 — FINAL NOTES & LAUNCH CHECKLIST

Pre-Launch Checklist

Legal & Compliance

- [] README includes full attribution to open-source dependencies
- [] Privacy policy live at `/privacy`
- [] Terms of service live at `/terms`
- [] GDPR data export + delete endpoints functional
- [] No IRIS trademarks or copyrighted content used

Technical

- [] All agent system prompts are 100% original
- [] BYOK flow tested: key enters client → header → agent → never persisted
- [] Rate limiting active on all auth + agent endpoints
- [] Error states handle API key failures gracefully (user-friendly message)
- [] Streaming SSE reconnects on dropped connections
- [] Mobile view tested at 375px (iPhone SE)
- [] Dark mode + light mode both functional

Product

- [] Onboarding flow: user can go from sign-up → first agent response in < 3 minutes
- [] Empty states are designed (not blank screens)
- [] At least one "wow" moment in the UX (live streaming tokens, agent identity reveal)
- [] Community channels live (Discord or GitHub Discussions)

Signhify AI — Type less. Signhify everything.

